

HORUS-LAB · ÉQUIPE TECHNIQUE

Bonnes pratiques Git & GitHub

Le guide de référence pour travailler à plusieurs sur nos dépôts, sans casser la production. Conventions, workflow quotidien, revue de code et procédures de secours.

HORUS-LAB

Solutions technologiques intelligentes

Version 2 · Juillet 2026

Dépôt : [horus-lab-team-s/website-horus-lab](#)

1. Les 8 règles d'or
2. Vocabulaire essentiel
3. Configuration initiale
4. Modèle de branches
5. Écrire de bons commits
6. Le workflow quotidien
7. Les Pull Requests
8. La revue de code
9. Résoudre un conflit
10. Sécurité : secrets et fichiers
11. Notre chaîne CI/CD
12. Commandes de secours
13. Checklist avant chaque PR

#	Règle	Pourquoi
1	On ne pousse jamais directement sur <code>main</code>	<code>main</code> est la branche de production : tout passe par une branche + Pull Request.
2	Une branche = une intention	Une PR qui fait 3 choses est impossible à relire et à annuler proprement.
3	Aucun secret dans Git	Un mot de passe poussé une fois reste dans l'historique pour toujours .
4	On met à jour avant de commencer	<code>git pull --rebase</code> évite 90 % des conflits.
5	Des commits petits et lisibles	Un commit = un changement cohérent, expliqué en une phrase.
6	Le code doit compiler avant le push	On ne délègue pas ses erreurs à l'équipe ni à la CI.
7	Pas de <code>--force</code> sur une branche partagée	Cela réécrit l'histoire des autres et détruit leur travail.
8	On relit les PR des autres	La revue est le seul filet de sécurité avant la production.

2 Vocabulaire essentiel

Terme	Définition simple
Dépôt (repo)	Le projet et tout son historique. Le nôtre est un monorepo : frontend/ (Next.js) et backend/ (Django) dans un seul dépôt.
Commit	Une photo d'un ensemble de modifications, avec un message qui explique le <i>pourquoi</i> .
Branche	Une ligne de travail isolée. On y développe sans risquer de casser main .
main	La branche de référence. Ce qu'elle contient part en production.
Remote / origin	Le dépôt distant sur GitHub. origin en est le nom par défaut.
Pull Request (PR)	Une demande d'intégration de votre branche dans main , ouverte à la relecture.
Merge	L'intégration effective de la PR dans main .
Conflit	Deux personnes ont modifié les mêmes lignes : Git demande un arbitrage humain.

3 Configuration initiale

À faire **une seule fois** sur votre machine. Utilisez vos vrais nom et e-mail : c'est ce qui apparaîtra sur chaque commit.

```
# Identité (obligatoire)
git config --global user.name "Prénom Nom"
git config --global user.email "vous@horus-lab.com"

# Confort et sécurité
git config --global init.defaultBranch main
git config --global pull.rebase true      # historique linéaire, moins de merges parasites
git config --global core.autocrlf true    # Windows : gère les fins de ligne
```

Récupérer le projet

```
git clone https://github.com/horus-lab-team-s/website-horus-lab.git
cd website-horus-lab
```

Authentification

GitHub n'accepte plus le mot de passe de compte. Utilisez un **Personal Access Token** (Settings → Developer settings → Tokens) ou une **clé SSH**. Un token se traite comme un mot de passe : jamais dans le code, jamais dans un message de chat.

4

Modèle de branches

Nom de branche = **préfixe** + **tiret bas de description courte**, en minuscules, mots séparés par des tirets.

Préfixe	Pour quoi	Exemple
feat/	Nouvelle fonctionnalité	feat/page-contact
fix/	Correction de bug	fix/favicon-manquant
refactor/	Réorganisation sans changer le comportement	refactor/composant-header
chore/	Outillage, dépendances, config	chore/maj-dependances
docs/	Documentation seule	docs/guide-git

✓ À faire

- feat/carrousel-temoignages
- fix/logo-invisible-navbar

✗ À éviter

- test, modif, branche-loic
- nouvelle_version_finale_2

5 Écrire de bons commits

Nous suivons la convention **Conventional Commits**. Le message répond à la question : « que fait ce changement, et pourquoi ? »

`type(portée): résumé à l'impératif, minuscule, sans point final`

Corps optionnel : le POURQUOI, le contexte, les effets de bord.
Une ligne vide sépare le titre du corps.

Type	Usage
<code>feat</code>	Ajoute une fonctionnalité visible par l'utilisateur
<code>fix</code>	Corrige un bug
<code>refactor</code>	Change le code sans changer le comportement
<code>style</code>	Mise en forme, sans impact fonctionnel
<code>docs</code>	Documentation
<code>chore</code>	Config, dépendances, scripts

✓ Bons messages

- `feat(contact):` ajoute la page Contactez-nous
- `fix(navbar):` corrige l'inversion des logos clair/sombre
- `chore(ci):` passe le build Node en version 20

✗ Mauvais messages

- `update · fix bug · wip`
- `modifications diverses`
- `ça marche enfin !!!`

Bonne granularité

Un commit doit pouvoir être annulé seul sans rien casser d'autre. Si votre message contient « et », c'est probablement deux commits.

6 Le workflow quotidien

Ces six étapes couvrent 95 % du travail. Répétez-les pour chaque tâche.

```
# 1. Repartir d'un main à jour
git checkout main
git pull --rebase origin main

# 2. Créer sa branche
git checkout -b feat/ma-fonctionnalite

# 3. Travailler, puis vérifier ce qu'on s'apprête à committer
git status
git diff

# 4. Committer par petits lots cohérents
git add frontend/src/components/Header.tsx
git commit -m "feat(navbar): ajoute le menu latéral mobile"

# 5. Publier la branche
git push -u origin feat/ma-fonctionnalite

# 6. Ouvrir la Pull Request sur GitHub, puis attendre la relecture
```

Avant chaque push

Vérifiez que le projet compile : `npm run build` dans `frontend/`. Un push qui casse le build bloque toute l'équipe.

Rester à jour pendant un travail long

```
# Récupérer les évolutions de main sans polluer l'historique
git fetch origin
git rebase origin/main
```

7 Les Pull Requests

La PR est le moment où l'équipe voit votre travail. Elle doit se relire en **moins de 20 minutes**.

Une bonne description contient

- **Le problème** : ce qui n'allait pas, ou le besoin.
- **La solution** : ce que vous avez changé, et pourquoi ainsi.
- **Comment tester** : les étapes concrètes pour vérifier.
- **Captures d'écran** pour tout changement visuel (avant / après).
- **Les risques** ou points qui méritent l'attention du relecteur.

Taille de la PR	Qualité de la relecture
< 200 lignes	Excellente — relue sérieusement
200 à 500 lignes	Correcte — acceptable
> 1000 lignes	Illusoire — approuvée sans être vraiment lue

Ne jamais merger si...

la CI est rouge · des commentaires de revue sont sans réponse · vous n'avez pas testé vous-même · la PR contient des fichiers sans rapport avec son objet.

8 La revue de code

✓ Relecteur

- Commentez le *code*, jamais la personne.
- Proposez une solution, pas seulement un reproche.
- Distinguez le bloquant du simple avis.
- Signalez aussi ce qui est bien fait.

✓ Auteur

- Une remarque vise le code, pas vous.
- Répondez à **chaque** commentaire.
- En désaccord ? expliquez, n'ignorez pas.
- Corrigez par de nouveaux commits.

Formulations utiles : « *Que penses-tu de... ?* », « *Bloquant* : ceci plantera si la liste est vide », « *Détail* : on pourrait renommer cette variable ».

Un conflit n'est pas une erreur : Git vous demande simplement d'arbitrer.

```
<<<<<< HEAD
const titre = "Nos services";      // votre version
=====
const titre = "Nos prestations";   // celle de main
>>>>>> origin/main
```

1. Ouvrez le fichier et gardez la version correcte (parfois une fusion des deux).
2. Supprimez les marqueurs <<<, ==, >>>.
3. Testez que le projet compile toujours.
4. Puis :

```
git add le-fichier-resolu.tsx
git rebase --continue      # si vous étiez en rebase

# En cas de doute, on annule et on recommence :
git rebase --abort
```

Le bon réflexe

Dans le doute sur le code d'un collègue, demandez-lui. Un message de 30 secondes vaut mieux qu'un choix arbitraire qui supprime son travail.

Règle absolue

Aucun mot de passe, token, clé d'API ou fichier `.env` ne doit entrer dans Git. Supprimer le fichier au commit suivant **ne suffit pas** : la valeur reste dans l'historique et doit être considérée comme compromise.

Ne jamais committer	À la place
<code>.env</code> , <code>stack.env</code> , secrets	Un fichier <code>.env.example</code> sans les valeurs
Tokens GHCR / clés SSH	GitHub Secrets (Settings → Secrets)
<code>node_modules/</code> , <code>.venv/</code> , <code>.next/</code>	Le <code>.gitignore</code> du dépôt
Dossiers d'outils locaux (<code>.claude/</code> , <code>.idea/</code>)	Le <code>.gitignore</code>
Gros binaires non optimisés	Images compressées, ou stockage externe

Si un secret a été poussé

1. **Révoquez-le immédiatement** (régénérez le token / changez le mot de passe).
2. Prévenez l'équipe.
3. Ensuite seulement, nettoyez l'historique.

Le réflexe qui sauve

Toujours lancer `git status` avant `git add`, et se méfier de `git add .` / `git add -A` qui embarquent tout, y compris ce que vous n'aviez pas vu.

Comprendre ce qui se passe après le merge évite les mauvaises surprises.

```
Branches → Pull Request → Revue → Merge dans main
      ↓
    GitHub Actions construit les images Docker
      ↓
  Publication sur GHCR (registre GitHub)
      ↓
  Déploiement sur le VPS (pull & up -d)
```

Ce que vous modifiez	Conséquence
<code>frontend/**</code>	Reconstruit l'image horus-frontend
<code>backend/**</code>	Reconstruit l'image horus-backend
Documentation seule	Aucun rebuild d'image

Deux pièges maison

1. Le merge ne déploie pas toujours tout seul : selon la configuration, il faut lancer le `pull & up -d` sur le serveur.
2. Vérifiez `RUN_SEED=0` avant un redéploiement, sinon le script de démarrage **réécrit les contenus saisis dans l'admin**.

12 Commandes de secours

Situation	Commande
Annuler des modifications non committées	<code>git restore fichier.tsx</code>
Retirer un fichier du <i>staging</i>	<code>git restore --staged fichier.tsx</code>
Mettre son travail de côté	<code>git stash</code> puis <code>git stash pop</code>
Corriger le dernier commit non poussé	<code>git commit --amend</code>
Annuler un commit déjà poussé	<code>git revert <hash></code>
Revoir l'historique	<code>git log --oneline --graph -15</code>
Retrouver un travail « perdu »	<code>git reflog</code>
Récupérer un commit précis d'une autre branche	<code>git cherry-pick <hash></code>

Attention

`git reset --hard` et `git push --force` détruisent du travail. Sur une branche partagée, préférez toujours `git revert`, qui annule proprement sans réécrire l'histoire des autres.

Rien n'est vraiment perdu

Tant qu'un travail a été commité au moins une fois, `git reflog` permet presque toujours de le retrouver. Respirez, puis demandez de l'aide.

✓	Point à vérifier
☐	Ma branche part d'un <code>main</code> à jour
☐	Le projet compile (<code>npm run build</code>) et j'ai testé mon changement
☐	Aucun secret, <code>.env</code> ou fichier local dans le diff
☐	Mes messages de commit respectent la convention
☐	Le diff ne contient que ce qui concerne cette tâche
☐	Pas de code mort, ni de <code>console.log</code> oublié
☐	La description explique le problème, la solution et le test
☐	Captures d'écran jointes si l'interface change
☐	La CI est verte

L'esprit du guide

Ces règles n'existent pas pour ralentir, mais pour que chacun puisse reprendre le travail d'un autre sans crainte. Un dépôt clair aujourd'hui, c'est une équipe rapide demain.